PRESEARCH.md

FleetGraph — Pre-Search Document

Completed before writing any code

Phase 1: Define Your Agent

1. Agent Responsibility Scoping

What events in Ship should the agent monitor proactively?

- A document is saved (webhook trigger — primary, satisfies <5 min latency requirement — existence confirmed as engineering spike #1)
- Nightly scheduled scan of all projects (poll trigger — safety net, catches anything the webhook missed)
- An issue moves to in-progress without sufficient acceptance criteria or context
- An active sprint has had no standup activity when the reasoning node determines one is expected

What constitutes a condition worth surfacing?

- A quality score below a defined threshold at any layer (project, sprint, issue, standup)
- An issue marked as started where required fields are incomplete or dependent information is missing
- A blocking issue that has not been resolved and is holding up downstream work
- A document under a person's ownership that has insufficient content to act on
- A director has submitted feedback on a quality report that the document owner has not yet seen
- Note: staleness is not time-based. It is an output of the reasoning node. If the analysis finds a document coherent and complete, it is not stale regardless of age. Staleness is a symptom the agent assigns when it detects broken relationships, missing information, or absent expected activity.

What is the agent allowed to do without human approval?

- Score any document or document graph and write the result to the document's `properties` JSONB field via REST API

- Generate and cache a plain text summary of a document and store it in `properties`
- Write quality score tags to documents
- Surface alert tags via Ship's existing accountability patterns when a user opens a document
- Generate pre-filled director response options for review
- Generate ephemeral `>>` inline prompt results inside a document (proposed UX addition — not existing Ship capability)

What must always require confirmation?

- Publishing a quality report as a new `quality_report` document to the project graph
- Applying a `>>` inline prompt result permanently to a document
- Sending director feedback to a document owner (director must select and confirm a pre-filled response)

How does the agent know who is on a project?

Ship's existing auth model is used directly via REST API — no direct DB access:

- `workspace_memberships.role = admin` → treated as Director
- Document `owner_id` → treated as PM for that document
- All other `workspace_memberships.role = member` → treated as Engineer

No database changes required for MVP. Role differentiation beyond ownership is post-MVP scope.

How does the agent know who to notify?

The agent does not push notifications. It writes alert tags to document `properties` and links each document to its parent quality report. Ship's existing accountability surfacing patterns are used to show the alert when a user opens a document. The exact UI surface for proactive findings is engineering spike #2.

How does the on-demand mode use context from the current view?

The on-demand mode is an inline prompt engine embedded directly inside every Ship document. *This is a proposed UX addition, not an existing Ship capability.* The user types `>>` anywhere in the document body to invoke it. The full document content, its parent project context, its current quality score from `properties`, and its linked quality report are all injected as a system prompt. The user types a freeform question. The result appears as an ephemeral inline block. The user can dismiss it or promote it to permanent document content.

If the `>>` approach carries too much product risk for MVP, the fallback is a scoped sidebar chat panel attached to the document context — same graph, lower-risk UI surface. This is engineering

spike #3.

---

2. Use Case Discovery

#	Role	Trigger	Agent Detects / Produces	Human Decides
1	PM	Nightly poll or document save webhook fires on a project	Agent traverses the full document graph (using `document_associations` plus parent-child relationships), scores each layer for completeness and quality using cached `properties` summaries where unchanged, produces a draft quality report with per-layer scores, traffic light status, and ranked remediation suggestions	PM reviews the draft quality report and confirms before it is published as a new `quality_report` document in the project graph
2	Engineer	An issue is moved to in-progress	Agent checks the issue for required fields: acceptance criteria, assignee, linked sprint, sufficient description. If any are missing, agent writes a "not ready" alert tag to the document `properties` and links to the quality report flagging the gap	Engineer sees the alert on next open; decides whether to proceed or request the missing information from the owner
3	PM	Nightly poll detects issues marked as blockers that have had no activity	Agent identifies unresolved blocking issues across the project graph, scores their impact on downstream documents via `document_associations`, and adds an alert tag to each blocked document and to the sprint document	PM reviews the flagged blockers in the quality report and decides which to action, reassign, or escalate
4	Engineer / PM	Nightly poll or document save on a document where `owner_id` matches a person	Agent checks all documents owned by a person for content depth — thin descriptions, missing context, unfilled fields. Scores each and writes alert tags to the thin documents via `properties`	Owner sees alerts on next open and decides what information to add
5	Director	Quality report is published to a project	Agent generates 3–5 pre-filled response options for the director based on the quality score and specific gaps identified. Director selects or edits one. Agent writes	Director selects a pre-filled response and confirms. Engineer or PM sees the director

#	Role	Trigger	Agent Detects / Produces	Human Decides
			the feedback as an alert tag on the relevant documents	feedback alert on next open of that document

3. Trigger Model Decision

Decision: Hybrid — webhook primary, nightly poll as safety net

Webhook (primary):

- Fires on document save events in Ship
- Triggers the graph immediately on content change
- The only mechanism capable of satisfying the <5 minute detection latency requirement
- Debounce required: agent checks `last_scored_at` in document `properties` before running. If the only change since last run was agent-written data, the graph does not re-run
- Summary hash comparison prevents tight loops from agent writes triggering re-scores
- Whether a usable save webhook exists in Ship is engineering spike #1

Nightly poll (safety net):

- Runs once per night as a scheduled Railway cron job
- Catches absence-based conditions that produce no save event (e.g. active sprint with no standups, in-progress issue with no updates)
- Scores the full project graph for all active projects
- Uses cached summaries in `properties` where documents have not changed, minimizing token cost

Tradeoffs considered:

Option	Latency	Cost	Complexity	Reliability
Poll only	High (up to 24hrs)	Predictable	Low	Misses latency window
Webhook only	Low (<5 min)	Variable	Medium	Single point of failure
Hybrid	Low (<5 min)	Predictable + variable	Medium	Best coverage

On staleness: Staleness is not a time-based concept in FleetGraph. There is no fixed timer after which a document is considered stale. Staleness is an output of the reasoning node — if the

analysis finds a document coherent and complete, it is not stale regardless of when it was last touched. If the analysis detects broken relationships, missing information, or absence of expected activity given the document type and project state, that is the symptom the agent flags. The nightly poll catches these absence-based conditions since they produce no save event to trigger the webhook.

Estimated runs per day:

- Webhook triggers: ~5 per active project per day (assumption: average document save frequency on an active project)
- Nightly poll: 1 per project per day
- Total: ~6 runs per project per day
- On-demand `>>` inline prompts: ~2 per user per day (assumption)

Cost at scale:

- Per run: ~$0.018 full scan, ~$0.004 cached summary scan
- Real-world mix assumed: ~70% cached, ~30% full scan → blended ~$0.008/run
- 100 projects: ~6 runs/day × 100 × $0.008 → ~$145/month
- 1,000 projects: ~$1,450/month
- 10,000 projects: ~$14,500/month
- Real-world costs estimated 60-70% lower than worst case due to summary caching

---

Phase 2: Graph Architecture

4. Node Design

Context nodes:

- Identify trigger source (webhook vs poll vs on-demand user invocation)
- Extract invoking user, document being viewed, project scope via Ship REST API
- Check `last_scored_at` in document `properties` — abort if only agent metadata has changed
- Determine role from Ship REST API (admin = Director, owner_id = PM, member = Engineer)
- For on-demand: extract current document context, quality score, and linked quality report from `properties` to inject into system prompt

Fetch nodes (run in parallel):

- Fetch project document (goals, description, `properties`, owner) via REST
- Fetch sprint documents (scope, dates, status, linked issues) via REST
- Fetch issue documents (acceptance criteria, assignee, labels, blockers) via REST
- Fetch standup documents (frequency, substance, recency) via REST
- Fetch `document_associations` to traverse cross-document relationships — data model is a graph, not only parent-child hierarchy
- Each fetch node checks `properties` for cached summary + last modified timestamp
- If summary exists and document is unchanged → use cached summary (short path)
- If document is new or modified → fetch full content, strip TipTap JSON to plain text (confirmed: Ship stores TipTap JSON)

Reasoning node (OpenAI SDK — GPT-4o):

- Note: Anthropic/Claude is not available on the government track. GPT-4o via OpenAI SDK is the approved model for this implementation.
- Receives full compressed project graph as single payload
- Scores each layer: project, sprint, issue, standup
- Detects gaps, missing fields, absence-based staleness, blocking issues, thin ownership documents
- Returns structured JSON: per-document scores, traffic light status, ranked remediation suggestions, pre-filled director response options
- Generates or updates cached plain text summary for each document

On-demand graph path (explicit):

- Context node injects current document + related graph nodes + existing quality score from `properties`
- Fetch nodes pull related documents in parallel via `document_associations`
- Reasoning node answers the user's freeform question OR detects an actionable condition
- Conditional edge: if answer-only → output ephemeral result inline; if action proposed → route to HITL gate
- HITL gate: user decides to dismiss ephemeral result or promote it to permanent document content

Conditional edges:

- If any layer scores below threshold → route to action node (low score path)
- If all layers score above threshold → route to output node (high score path)
- If Ship API fails → route to error/fallback node
- On-demand: if reasoning produces a proposed action → route to HITL gate; if answer-only → route directly to output

Action nodes:

- Draft quality report document of type `quality_report` (full type plumbing scope is engineering spike #4)
- Write score, summary, and `last_scored_at` to document `properties` JSONB for each document in graph via REST
- Write alert tags to documents that have gaps
- Link each document to its parent quality report via `properties`
- Generate pre-filled director response options

Human-in-the-loop gates:

- PM confirms before quality report document is published to project graph
- Director confirms before feedback is written to document owner's alert tags
- User confirms before `>>` inline prompt result is promoted to permanent document content

Output nodes:

- Publish quality report as new `quality_report` document in Ship project graph
- Write score badge and traffic light status to `properties` on all scored documents
- Write alert tags to flagged documents (surfaced via Ship's existing accountability patterns on next open)
- All-clear path: update scores and summaries in `properties` only, no report published

Error and fallback nodes:

- Ship API down: log failure, skip run, do not write partial scores to `properties`
- Missing data: score available layers, flag missing layers explicitly in report
- Unexpected document state: degrade gracefully, do not crash graph

---

5. State Management

State carried across a single graph run:

- Trigger source and invoking context
- Project ID and full document graph structure (via `document_associations`)
- Fetch results for all parallel nodes
- Extracted plain text content per document (stripped from TipTap JSON)
- Scores and gap analysis per layer
- Generated quality report draft

State persisted between runs (written to document `properties` JSONB via Ship REST API):

- `quality_score`: numeric score per document
- `quality_status`: traffic light value (green / yellow / red)
- `quality_summary`: compressed plain text summary of document content
- `last_scored_at`: ISO timestamp of last successful score run
- `quality_report_id`: link to the parent quality report document
- `quality_tags`: array of alert tags written by the agent

Avoiding redundant API calls:

- Webhook trigger checks `last_scored_at` and summary hash in `properties` before fetching full content
- Fetch nodes use cached summary if document is unchanged since last run
- Single compressed payload sent to reasoning node — not one call per document
- `document_associations` fetched once per run and reused across all fetch nodes

---

6. Human-in-the-Loop Design

Which actions require confirmation?

1. Publishing a quality report document to the project graph (PM confirms)
2. Director sending feedback to a document owner (Director selects and confirms pre-filled response)
3. Promoting a `>>` inline prompt result to permanent document content (user confirms inline)

What does the confirmation experience look like in Ship?

- For quality report publishing: agent surfaces the draft report with a publish/dismiss action — exact UI surface determined by engineering spike #2
- For director feedback: director sees pre-filled response options, selects one, confirms before it is written as an alert tag to `properties`
- For `>>` inline results (proposed UX): ephemeral block appears inline with "Add to document" and "Dismiss" controls

What happens if the human dismisses or snoozes?

- Quality report draft is discarded; scores and metadata are still written to `properties`
- Director feedback is not sent; no alert tag written
- `>>` result is removed from the document; no permanent change

---

7. Error and Failure Handling

Ship API is down:

- Graph catches the error at the fetch node level
- Logs the failure with timestamp
- Skips the run entirely — does not write partial scores to `properties`
- Nightly poll retries on next scheduled run
- Webhook retries on next document save event

Graceful degradation:

- If one fetch node fails but others succeed, the graph scores available layers and explicitly flags missing layers in the quality report
- Partial scores are marked as incomplete so humans are not misled

Caching:

- Document summaries cached in `properties` JSONB — valid until document content is modified
- `last_scored_at` timestamp used to determine cache validity
- No external cache layer required for MVP; `properties` is the cache store

---

Phase 3: Stack and Deployment

8. Deployment Model

Where does the proactive agent run when no user is present?

- Deployed as a persistent Node.js service on Railway
- Two processes: a webhook listener (always-on HTTP server) and a nightly cron job (scheduled Railway job)

How is it kept alive?

- Railway persistent service with automatic restart on failure
- Webhook listener is stateless — each incoming webhook triggers a fresh graph run

How does it authenticate with Ship without a user session?

- Ship API token stored as Railway environment variable
- Agent uses existing API token auth mechanism in Ship's auth middleware
- No user session required; agent acts as a system-level actor
- All data access through Ship REST API only — no direct database access

Environment variables required:

- `SHIP_API_URL` — base URL of the deployed Ship instance
- `SHIP_API_TOKEN` — service-level API token for Ship
- `OPENAI_API_KEY` — GPT-4o API key (OpenAI SDK)
- `LANGCHAIN_TRACING_V2=true`
- `LANGCHAIN_API_KEY` — LangSmith key for tracing

---

9. Performance

How does the trigger model achieve <5 minute detection latency?

- Webhook fires immediately on document save
- Graph run triggered within seconds of webhook receipt
- Debounce check at context node prevents unnecessary re-runs but adds no meaningful latency
- Nightly poll does not contribute to latency for time-sensitive detections

Token budget per invocation:

- Full project scan (no cache): ~4,000 tokens in, ~1,000 tokens out → ~$0.018/run at GPT-4o pricing
- Cached summary scan: ~800 tokens in, ~500 tokens out → ~$0.004/run
- On-demand `>>` inline prompt: ~2,000 tokens in, ~500 tokens out → ~$0.008/invocation
- Actual token counts to be validated against real Ship data after TipTap serialization spike

Cost cliffs:

- Large projects with many issues and no cached summaries hit the high end of token budgets
- Webhook debounce failure would multiply costs — summary hash check in `properties` is the critical guard
- At 1,000 active projects: ~$4/night cached rate, ~$18/night full scan rate

---

10. Known Unknowns — Engineering Spikes (Resolve Before Tuesday)

1. **Does Ship expose a usable document save webhook?** Primary spike. If not, webhook trigger must be added or the hybrid model falls back to poll-only for MVP.
2. **What is the best existing UI surface for proactive findings?** Ship has existing accountability surfacing patterns. FleetGraph should use these rather than inventing new UI where possible.
3. **How should on-demand chat be embedded with minimal product risk?** The `>>` inline prompt is the target design but carries editor integration risk. Scoped sidebar panel is the fallback. Must be decided before building.
4. `quality_report` **document type plumbing** — Adding a new document type requires more than one enum migration. Full scope to be assessed before building.